

МИНОБРНАУКИ РОССИИ
ФГАОУ ВО «МГТУ «СТАНКИН»
Кафедра Центр подготовки топ-специалистов в сфере ИТ ЛЕКЦИЯ №1

Программирование и инструменты Индустрии 3.0

по дисциплине
«Технологии разработки программных средств» **Автор:** Семенов А.С.

Москва, 2026 г.

Аннотация

Лекция посвящена первому этапу эволюции разработки ПО — Индустрии 3.0 (эпоха автоматизации). Рассматриваются низкоуровневые языки программирования (машинные коды, ассемблер, Fortran, C, Pascal), инструментарий того времени (редакторы, компиляторы, отладчики, CASE-средства), монолитная архитектура и ручные методы тестирования. Показана связь между уровнем абстракции языков и производительностью труда, а также ограничения, которые привели к кризису ПО и необходимости перехода к следующему этапу. Лекция содержит временные оси, схемы и примеры из реальной практики.

Ключевые слова

Индустрия 3.0, машинные коды, ассемблер, языки высокого уровня, компиляторы, монолит, структурное программирование, CASE-средства.

Содержание

Введение: контекст Индустрии 3.0	3
Эволюция языков программирования в 3.0	3
Машинные коды и ассемблер (1950–1960-е)	3
Языки высокого уровня (1960–1980)	3
Парадигма программирования	3
Инструментарий разработчика 3.0	4
Редакторы и компиляторы	4
Системы контроля версий	4
CASE-средства (Computer-Aided Software Engineering)	4
Тестирование	4
Архитектура и организация кода	4
Монолитная архитектура	4
Структурное программирование	4
Пример: программа управления производственным станком	4
Ограничения и «кризис ПО»	5
Заключение и связь с курсом	5

Введение: контекст Индустрии 3.0

Индустрия 3.0 — это эпоха, которая началась с внедрения компьютеров и программируемых логических контроллеров (PLC) в производство. Главная цель ПО того времени — **автоматизация** повторяющихся операций. Разработка велась в условиях:

- Ограниченных вычислительных ресурсов (память, тактовая частота).
- Отсутствия стандартизированных процессов и инструментов.
- Жёсткой привязки к конкретному оборудованию.

Ключевые особенности разработки в 3.0:

- Языки низкого уровня (близкие к «железу»).
- Монолитная архитектура — вся программа в одном исполняемом файле.
- Водопадный жизненный цикл (Waterfall).
- Ручное тестирование и документирование.

Эволюция языков программирования в 3.0

Машинные коды и ассемблер (1950–1960-е)

Первые программы писались непосредственно в машинных кодах — наборах чисел, которые процессор выполнял напрямую. Это требовало глубокого знания архитектуры конкретного компьютера. Ассемблер ввёл символические обозначения для команд и адресов, что немного упростило жизнь, но всё ещё оставался низкоуровневым.

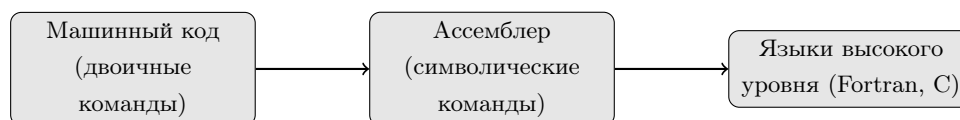


Рис. 1: Эволюция языков программирования в 3.0

Языки высокого уровня (1960–1980)

- **Fortran** (1957) — первый язык высокого уровня, ориентированный на научные и инженерные расчёты.
- **C** (1972) — язык системного программирования, дал возможность управлять памятью и аппаратурой, оставаясь достаточно абстрактным.
- **Pascal** (1970) — учебный язык, но также применялся в промышленности.

Эти языки позволили отвлечься от аппаратных деталей и сосредоточиться на алгоритмах. Однако программы всё ещё были близки к «железу» (управление указателями, ручное распределение памяти).

Парадигма программирования

Доминировало **процедурное (императивное) программирование**: программа — это последовательность инструкций (процедур и функций), изменяющих состояние данных. Структурное программирование (Бём и Якопини, 1966) добавило дисциплину: отказ от GOTO, использование циклов и условий.

Инструментарий разработчика 3.0

Редакторы и компиляторы

- Текстовые редакторы (ed, vi) без подсветки синтаксиса.
- Компиляторы (например, для C, Fortran) — отдельные программы, запускаемые из командной строки.
- Отладчики (например, gdb) — позволяли пошагово выполнять код, но без графического интерфейса.

Системы контроля версий

В 3.0 системы контроля версий практически не применялись. Код хранился в общих папках, изменения передавались устно или по электронной почте. Конфликты разрешались вручную, что было крайне неэффективно.

CASE-средства (Computer-Aided Software Engineering)

В конце 1980-х появились первые инструменты, помогающие в анализе и проектировании. Например:

- **DFD-редакторы** для построения диаграмм потоков данных.
- **ER-диаграммы** для моделирования данных.
- **Генераторы кода** — могли создавать заготовки программ по спецификациям.

Однако они были дорогими, сложными в настройке и не всегда давали нужный результат.

Тестирование

Тестирование проводилось вручную: разработчик запускал программу с набором тестовых данных и анализировал вывод. Автоматизация тестов практически отсутствовала.

Архитектура и организация кода

Монолитная архитектура

Вся программа компилировалась в один исполняемый файл. Все компоненты (ввод/вывод, бизнес-логика, работа с базой данных) были тесно связаны. Это давало преимущество в простоте сборки, но создавало огромные проблемы с поддержкой и масштабированием.

Структурное программирование

Код разбивался на функции и процедуры, вызываемые друг из друга. Это позволяло улучшить читаемость и повторное использование, но жёсткая глобальная память и отсутствие инкапсуляции часто приводили к побочным эффектам.

Пример: программа управления производственным станком

В 1980-х программист писал код на C, который напрямую читал и записывал регистры контроллера. Изменение одного параметра (например, скорости подачи) требовало перекомпиляции и перепрошивки микросхем ПЗУ.

Ограничения и «кризис ПО»

К концу 1980-х стало очевидно, что подходы 3.0 не справляются с растущей сложностью проектов. Основные проблемы:

- Негибкость: изменения требований приводили к огромным затратам.
- Низкая производительность: большая часть времени уходила на отладку и интеграцию.
- Высокая стоимость ошибок: ошибка, обнаруженная на этапе эксплуатации, могла стоить компании миллионы.

Этот кризис стал движущей силой для перехода к Индустрии 4.0, где появились объектно-ориентированное программирование, итеративные методологии и распределённые системы управления версиями.

Заключение и связь с курсом

Лекция показала, что Индустрия 3.0 заложила основы дисциплины, но её инструменты и подходы были слишком жёсткими для решения новых задач. Следующий этап — Индустрия 4.0 — принесёт объектно-ориентированное программирование, микросервисы, Agile и DevOps, о чём мы поговорим в следующих лекциях.

Контрольные вопросы

1. Перечислите три поколения языков программирования, доминировавших в 3.0. В чём их отличие?
2. Что такое монолитная архитектура? Какие её недостатки?
3. Какие инструменты использовались для разработки в 3.0? Почему они были неэффективны?
4. В чём суть структурного программирования?
5. Почему возник «кризис программного обеспечения»?